



國立清華大學
NATIONAL TSING HUA UNIVERSITY

Deep Learning and Applications

Ching-I Huang



Binary Classification with Logistic Regression

- Posterior probability is modeled as

$$P(C_1 | \mathbf{x}; \mathbf{w}, b) = P_{w,b}(C_1 | \mathbf{x}_n) = \sigma(\mathbf{w}^T \mathbf{x} + b)$$

Find the maximum posterior probability that fits the training data

- Likelihood of data**

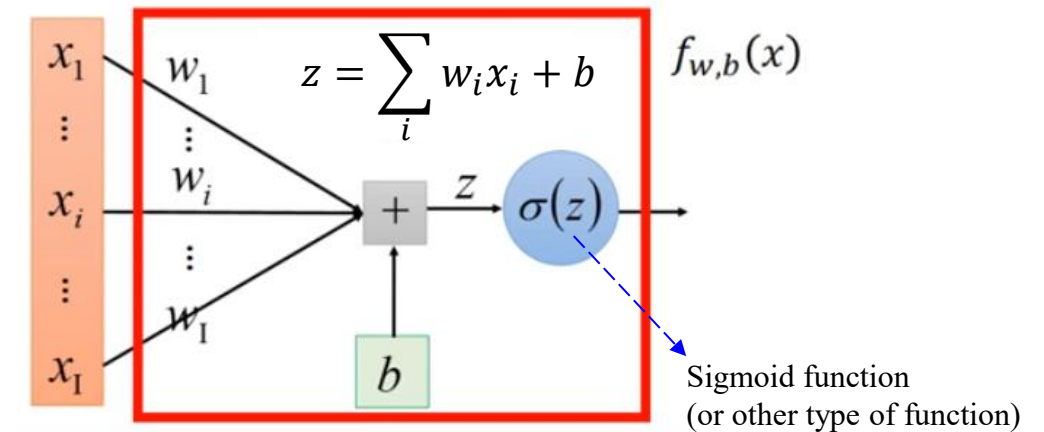
$$\begin{aligned} L_{w,b}(C_1 | \mathbf{x}_n) &= \prod_{n=1}^N \left[\frac{P_{w,b}(C_1 | \mathbf{x}_n)^{t_n}}{y_n} \frac{(1 - P_{w,b}(C_1 | \mathbf{x}_n))^{1-t_n}}{1 - y_n} \right] \\ &= \prod_{n=1}^N y_n^{t_n} (1 - y_n)^{1-t_n}, \quad y_n = \sigma(\mathbf{w}^T \mathbf{x}_n + b) \end{aligned}$$

- Training objective**

$$\mathcal{L} = - \sum_{n=1}^N [t_n \ln y_n + (1 - t_n) \ln(1 - y_n)]$$

Minimize the negative log-likelihood (cross-entropy loss)

$$\mathbf{w}^*, b^* = \arg \min_{\mathbf{w}, b} - \sum_{n=1}^N [t_n \ln y_n + (1 - t_n) \ln(1 - y_n)]$$



Training data

$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\dots$	$\mathbf{x}_n$
C_1	C_1	C_2	$\dots$	
$t_1=1$	$t_2=1$	$t_3=0$	$\dots$	t_n

$t_n = 1$ if $\mathbf{x}_n \in C_1$
 $t_n = 0$ if $\mathbf{x}_n \in C_2$



Binary Classification with Logistic Regression

Note that, the derivative of the logistic sigmoid function, which can conveniently be expressed in terms of the sigmoid function itself,

$$\frac{d\sigma}{dz} = \sigma(z)(1 - \sigma(z))$$

Taking the gradient of the error function w.r.t $\mathbf{w}$,

$$\nabla E_{\mathbf{w}} = \sum_n y_n \frac{\partial \log P(\mathcal{C}_1|\mathbf{x})}{\partial w_i} + (1 - y_n) \left(1 - \frac{\partial \log(1 - P(\mathcal{C}_1|\mathbf{x}))}{\partial w_i}\right)$$

where

$$\begin{aligned} \frac{\partial \log P(\mathcal{C}_1|\mathbf{x})}{\partial w_i} &= \frac{\partial \log P(\mathcal{C}_1|\mathbf{x})}{\partial z} \frac{\partial z}{\partial w_i} = \frac{\partial \log \sigma(z)}{\partial z} x_i \\ &= \frac{1}{\sigma(z)} \frac{\partial \sigma(z)}{\partial z} x_i = \frac{1}{\sigma(z)} \sigma(z)(1 - \sigma(z)) x_i = \boxed{(1 - \sigma(z)) x_i}, \\ \frac{\partial \log(1 - P(\mathcal{C}_1|\mathbf{x}))}{\partial w_i} &= -\frac{1}{1 - \sigma(z)} \frac{\partial \sigma(z)}{\partial z} x_i \\ &= -\frac{1}{1 - \sigma(z)} \sigma(z)(1 - \sigma(z)) x_i = \boxed{-\sigma(z) x_i}. \end{aligned}$$

Then,

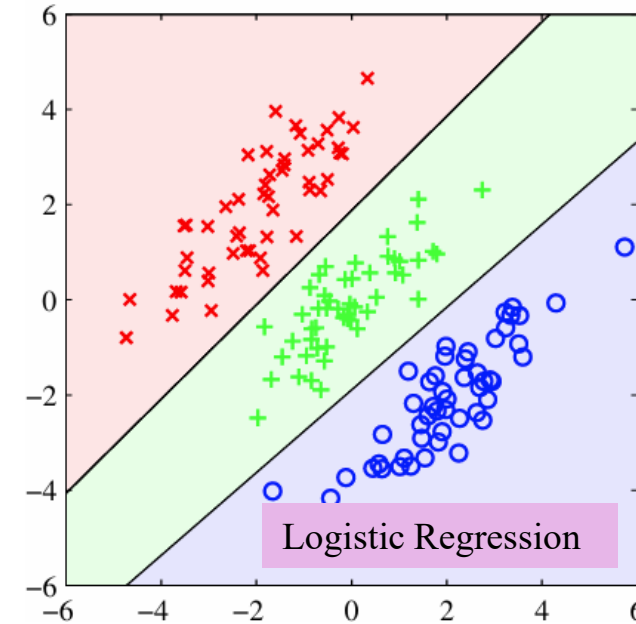
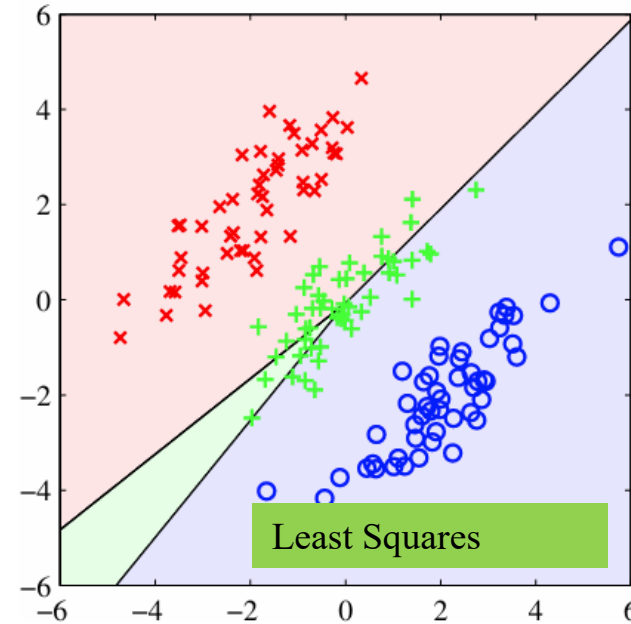
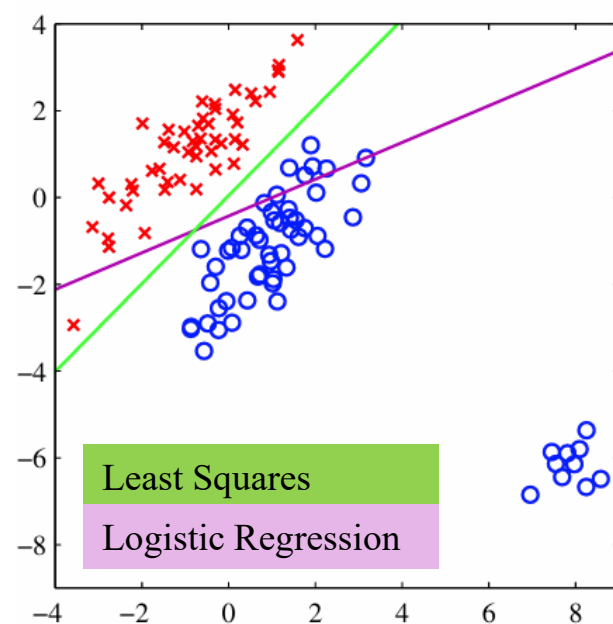
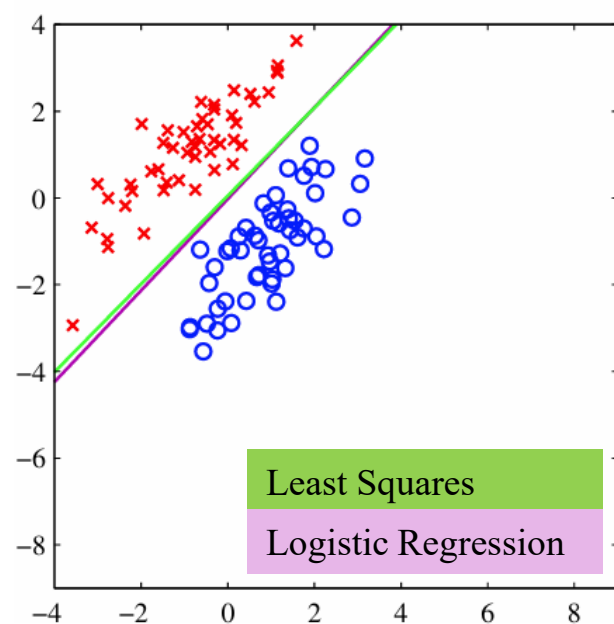
$$\begin{aligned} \nabla E_{\mathbf{w}} &= - \sum_n y_n (1 - \sigma(z)) x_i^n - (1 - y_n) \sigma(z) x_i^n \\ &= - \sum_n [y_n - y_n \sigma(z) - \sigma(z) + y_n \sigma(z)] x_i^n \\ &= - \sum_n (y_n - \sigma(\sum_i w_i^T x_i + b)) x_i^n \end{aligned}$$

Unlike in linear regression, there is no analytical solution $\mathbf{w}$ in a closed form.

Classification



- Classification is different task to regression.



- **Regression:** predict a numerical value given some input (fits values)

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^m \quad \text{e.g., temperature forecasting}$$

- **Classification:** predict a categorical label given some input (draws boundaries)

$$f : \mathbb{R}^n \rightarrow \{1, \dots, k\}$$

Multi-class Classification (4.2 and 4.3.4)

- When **K classes** ($K > 2$), assume that the target variable $\mathbf{t} \in \{0, 1\}^K$ is **one-hot encoded**, and follows a Categorical distribution:

$$p(\mathbf{t} \mid \mathbf{x}, \mathbf{w}) = \prod_{k=1}^K y_k(\mathbf{x})^{t_k} \quad \sum_{k=1}^K y_k(\mathbf{x}) = 1$$

- To ensure that the output vector $\mathbf{y} = (y_1, y_2, \dots, y_K)$ is a valid probability distribution, we need each $y_k \in (0, 1)$ and their sum to equal 1.

Softmax function

$$y_k = \frac{\exp(a_k)}{\sum_{j=1}^K \exp(a_j)}$$

The softmax function, also known as the *normalized exponential*, ensures that the model outputs form a valid probability distribution over the classes.

The MLE-optimal solution for modeling multi-class outputs probabilistically

$$P(\mathcal{C}_k \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid \mathcal{C}_k)P(\mathcal{C}_k)}{\sum_j^K P(\mathbf{x} \mid \mathcal{C}_j)P(\mathcal{C}_j)} = \frac{\exp(z_k)}{\sum_j^K \exp(z_j)}$$

$$z_k = \log P(\mathbf{x} \mid \mathcal{C}_k)P(\mathcal{C}_k) = \mathbf{w}_k^T \mathbf{x} + b_k$$



Multi-class Classification (4.2 and 4.3.4)

- The posterior probabilities for $K > 2$ classes are given by a softmax transformation of linear functions of the feature variables,
- When $z_k \gg z_j \forall j \neq k$, then $P(C_k | \mathbf{x}) \rightarrow 1$, which mimics the behavior of an argmax decision in classification.

$$P(C_k | \mathbf{x}) = \frac{P(\mathbf{x} | C_k) P(C_k)}{\sum_j^K P(\mathbf{x} | C_j) P(C_j)} = \frac{\exp(z_k)}{\sum_j^K \exp(z_j)}$$

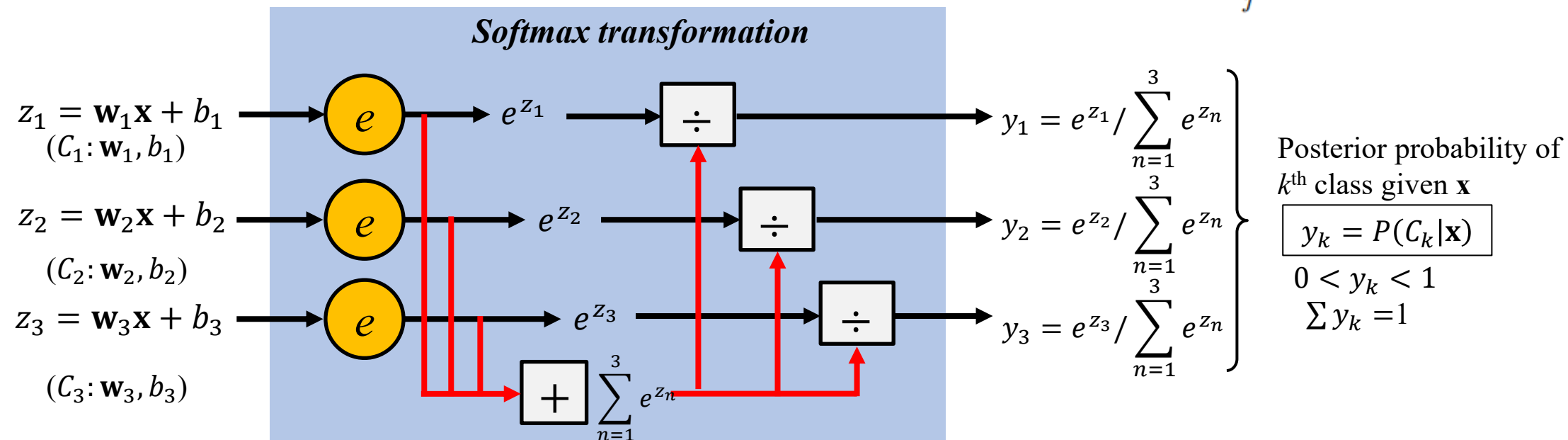
$$z_k = \log P(\mathbf{x} | C_k) P(C_k) = \mathbf{w}_k^T \mathbf{x} + b_k$$

$$e^{z_k} \gg e^{z_j} \quad \forall j \neq k$$

$$P(C_k | \mathbf{x}) \approx \frac{e^{z_k}}{e^{z_k}} = 1$$

$$P(C_j | \mathbf{x}) \approx 0 \quad \forall j \neq k$$

$$\hat{y} = \arg \max_j z_j$$

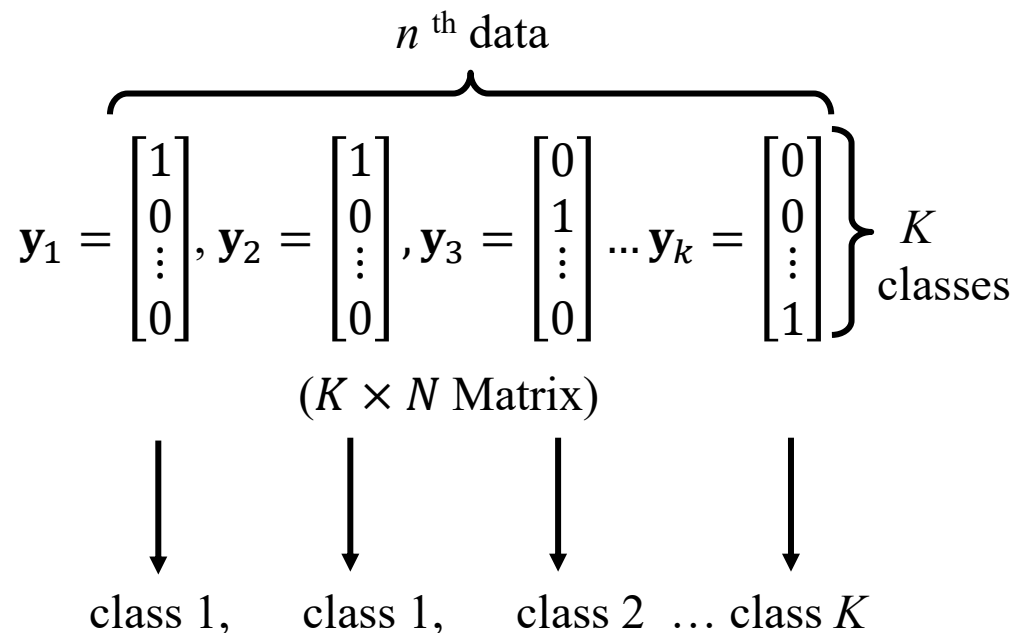




Multi-class Classification (4.2 and 4.3.4)

Training data

$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\dots$	$\mathbf{x}_N$
C_1	C_1	C_2	$\dots$	C_K



Prediction: $\hat{\mathbf{y}} \leftrightarrow$ Target: $\mathbf{y}$

one-hot encoding

$$\hat{\mathbf{y}} = [\hat{y}_1, \hat{y}_2, \dots, \hat{y}_K]$$

$$\mathbf{y} = [0, 0, \dots, 1, \dots, 0]$$

$$\mathbf{y}_n = [y_{n1}, \dots, y_{nK}]^T$$

- For each likelihood function that generates the data belonging to class k :

Prob. For class 1 $P(y_1 | \mathbf{w}_1, b_1) = \prod_{n=1}^N P(C_1 | \mathbf{x}_n)^{y_{n1}}$

Prob. For class 2 $P(y_2 | \mathbf{w}_2, b_2) = \prod_{n=1}^N P(C_2 | \mathbf{x}_n)^{y_{n2}}$

$\vdots$

$\vdots$

Prob. For class k $P(y_k | \mathbf{w}_K, b_K) = \prod_{n=1}^N P(C_K | \mathbf{x}_n)^{y_{nK}}$

where the target vector $\mathbf{y}_k$ belonging to C_k is a binary vector with all elements zero except for element k , which equals one, such $y_1 = [1; 0; \dots; 0]^T$, $y_2 = [0; 1; \dots; 0]^T$ and $y_K = [0; 0; \dots; 1]^T$.

The overall likelihood function is then given by

$$\begin{aligned} P(\mathbf{y} | \mathbf{w}_1, b_1, \dots, \mathbf{w}_K, b_K) &= \prod_{n=1}^N \prod_{k=1}^K P(C_k | \mathbf{x}_n)^{y_{nk}} \\ &= \prod_{n=1}^N \prod_{k=1}^K \hat{y}_{nk}^{y_{nk}} \end{aligned}$$

- The log-likelihood becomes:

$$\log \mathcal{L} = \sum_{n=1}^N \sum_{k=1}^K y_{nk} \log(\hat{y}_{nk})$$



Multi-class Classification (4.2 and 4.3.4)

- **Maximum likelihood estimation (MLE)** is equivalent to minimizing the cross-entropy loss

$$E_{\mathbf{w},b} = - \sum_{n=1}^N \sum_{k=1}^K y_{nk} \log \hat{y}_{nk}$$

- For one-hot labels,

$$E_{\mathbf{w},b} = -\log(\hat{y}_{\text{true class}})$$

Measures how well the predicted probability distribution approximates the true one.

- If true class = 2:

$$\hat{\mathbf{y}} = [0.1, 0.9, 0.0] \rightarrow \text{low loss}$$

$$\hat{\mathbf{y}} = [0.5, 0.5, 0.0] \rightarrow \text{higher loss}$$

$$\hat{\mathbf{y}} = [0.9, 0.1, 0.0] \rightarrow \text{very high loss}$$

- ✓ concentrate all probability mass on the correct class
- ✓ Sharp (confident) but accurate distributions.
- ✓ Penalizes both wrong predictions and uncertainty.
- ✓ Smooth and stable gradients

Although $\log(x) \rightarrow -\infty$ as $x \rightarrow 0$,
the softmax function ensure $0 < \hat{y}_k < 1$

- The gradient of the cross entropy loss w.r.t. the parameter vector $\mathbf{w}_j$

$$\nabla_{\mathbf{w}_j} E = \sum_{n=1}^N (\hat{y}_{nj} - y_{nj}) x_n$$

↑ ↑

Use Gradient descent method



Multi-class Classification (4.2 and 4.3.4)

- The overall likelihood function

$$P(\mathbf{y}|\mathbf{w}_1, b_1, \dots, \mathbf{w}_K, b_K) = \prod_{n=1}^N \prod_{k=1}^K P(C_k | \mathbf{x}_n)^{y_{nk}}$$

- Posterior Probability

$$y_{kn} = P(C_k | x_n, \mathbf{w})$$

For a given input x_n , the model predicts the probability of each class.

- Likelihood Function

$$L(\mathbf{w}) = \prod_{n=1}^N \prod_{k=1}^K P(C_k | x_n, \mathbf{w})^{t_{kn}}$$

With a given dataset, find parameters $\mathbf{w}$ such that the model's posterior probabilities best match the target.
(maximize the product of all posteriors over the dataset)



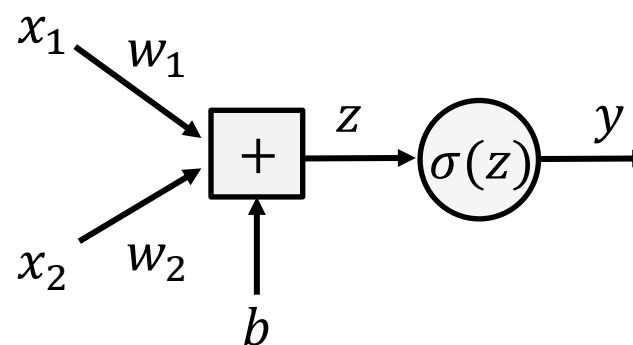
Concept of Neuron Network

- Limitation of logistic regression

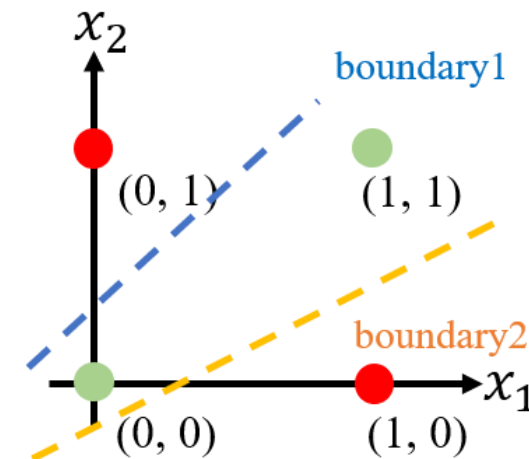
XOR

$$x = \left\{ \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right\}$$

$$y = f^*(x) \quad \begin{matrix} \text{A} & \text{B} & \text{C} & \text{D} \\ \text{0} & \text{1} & \text{1} & \text{0} \end{matrix}$$



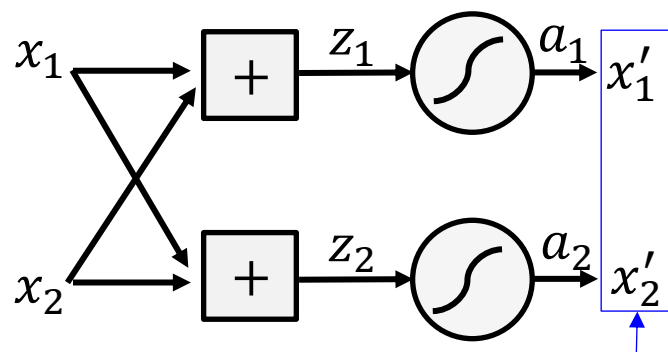
2-D input data x_1, x_2



Inseparable!

The boundary can be modified through adjusting the weighting w_1 and w_2 , but it cannot classify this case.

- Feature transformation



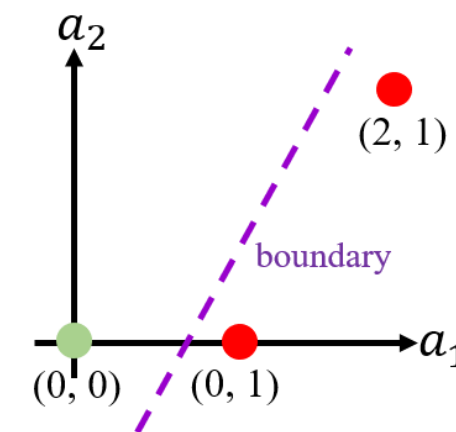
$$\sigma(z) = \max\{0, z\}$$

$$f(x; w_1, w_2, b_1, b_2) = w_2^T \max\{0, w_1^T x + b_1\} + b_2$$

$$w_1 = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, b_1 = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, w_2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, b_2 = 0$$

$$\begin{array}{l} \text{A: } \begin{bmatrix} z_1^{(1)} \\ z_2^{(1)} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \end{bmatrix} \rightarrow \begin{bmatrix} 0 \\ 0 \end{bmatrix} \\ \text{B: } \begin{bmatrix} z_1^{(1)} \\ z_2^{(1)} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \rightarrow \begin{bmatrix} 1 \\ 0 \end{bmatrix} \\ \text{C: } \begin{bmatrix} z_1^{(1)} \\ z_2^{(1)} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \rightarrow \begin{bmatrix} 1 \\ 0 \end{bmatrix} \\ \text{D: } \begin{bmatrix} z_1^{(1)} \\ z_2^{(1)} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} 2 \\ 1 \end{bmatrix} \end{array}$$

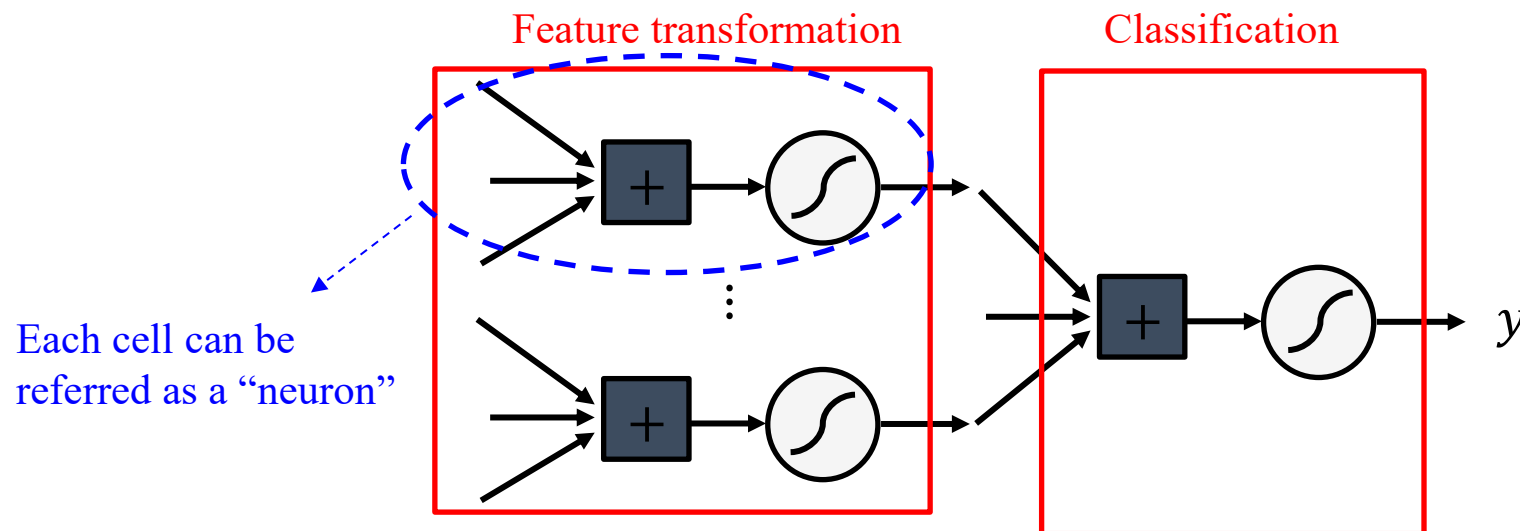
$$\begin{array}{l} y = \begin{bmatrix} 1 & -2 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} = 0 \\ y = \begin{bmatrix} 1 & -2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1 \\ y = \begin{bmatrix} 1 & -2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1 \\ y = \begin{bmatrix} 1 & -2 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = 0 \end{array}$$





Concept of Neuron Network

- Cascading logistic regression models to achieve binary classification



- In multi-class classification, feature transformation is usually multiple layers (Hidden layers).

